

University of Notre Dame

FPGA: HFT Exchange Market Making Over FPGA Process Documentation

Michael Agsam, Andrew Linares, Evan Bartek
CSE40438: High-Frequency Trading Technologies
Professor Matthew Belcher

Abstract:

This project explores the implementation of a high-frequency trading (HFT) strategy on the DE2-115 Altera Intel FPGA using the Quartus synthesis toolchain. Building on the success of a previous FPGA group, our team designed and tested hardware capable of reading market data from UDP packets and executing a simple trading strategy directly in hardware. The system parses incoming Ethernet packets, creates market data in the structure of a BUY order, and generates outgoing orders directly to the exchange. Additionally, we documented the full setup process for deploying the project on the DE2-115 board, including FPGA configuration, Ethernet interfacing, and debugging techniques. Challenges related to licensing restrictions, hardware limitations, and network configuration were addressed through the use of open-source tools and custom solutions. This project serves as a foundation for future FPGA-based trading systems that aim to directly interface with exchange infrastructure.

1 Project Proposal / Overview

For our final project, our group aimed to build upon the work of a previous FPGA team by designing a hardware-based trading system capable of directly interacting with the NDFEX. Unlike prior implementations that focused heavily on high-level synthesis tools via Xilinx, our approach emphasized practical deployment on the DE2-115 Altera Intel FPGA using the Quartus synthesis environment.

Our system reads incoming Ethernet packets containing market data (in the form of a trade summary), parses the UDP payload, and executes a simple trading strategy entirely in hardware. We are able to send out a hard-coded BUY order to the NDFEX in theory over ethernet.

1.1 Project Goals

The primary goals of this project were to implement hardware capable of parsing UDP-based market data directly from Ethernet packets, and to design a simple trading strategy that reacts to incoming market conditions. These two objectives formed the core of the system's functionality, ensuring that the FPGA could not only receive and interpret real-time market information but also respond to it intelligently based on predefined trading logic.

Alongside this, the project aimed to enable FPGA-generated order messages to be transmitted back over Ethernet, completing the full cycle of market data ingestion and order execution. To ensure accessibility and reproducibility, the project also prioritized providing clear and thorough documentation for setting up and running the system on the DE2-115 FPGA, allowing others to understand, replicate, and build upon the work.

1.2 Deliverables

- A guide for setting up the project on the DE2-115 Altera Intel FPGA using Quartus
 - A finite state machine (FSM) design for packet parsing and trading logic (provided separately)
 - Documentation on UDP market data deconstruction, including how payloads are parsed
 - Documentation on UDP payload construction for outgoing trading orders
 - A working demonstration using external software to simulate market data on Github
 - An example PCAP file showing the ethernet traffic along our local interface
-

2 System Overview

The system operates as a hardware-based trading pipeline:

1. Incoming Ethernet packets are received by the FPGA
2. UDP payloads are extracted and parsed to identify market data
3. A trading condition is evaluated based on parsed data
4. If conditions are met, a new UDP packet representing a trade order is constructed
5. The order is transmitted back over Ethernet

To maximize performance, these operations are designed to occur in a pipelined and hardware-efficient manner.

3 Trading Strategy Implementation

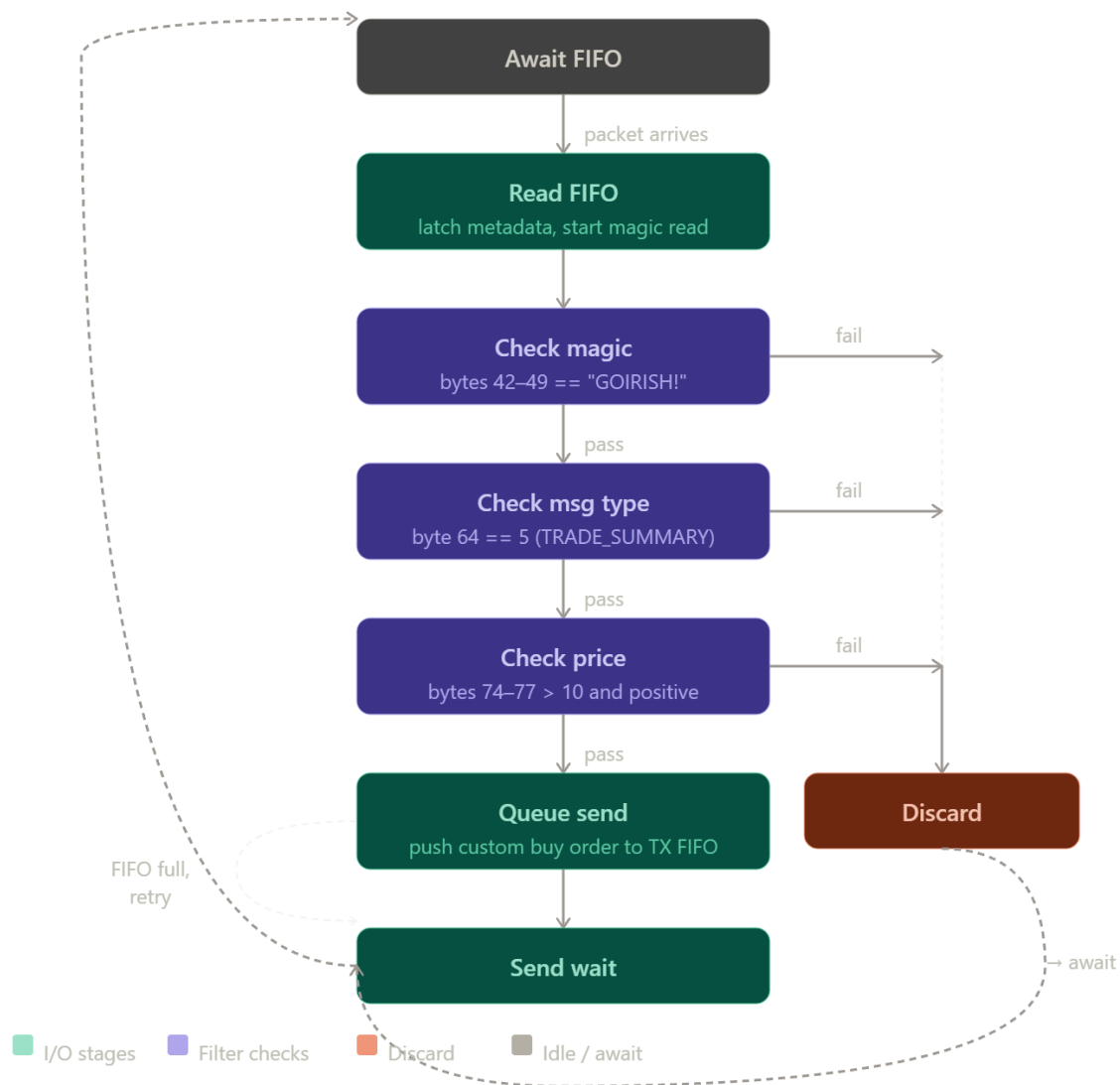
The implemented trading strategy is intentionally simple to validate system functionality:

- Market updates are received via UDP packets
- Each packet contains trade price information
- If the incoming trade price exceeds **\$10**, the FPGA generates a **buy order**
- Otherwise, no action is taken

This logic allows us to verify end-to-end functionality from packet reception to order transmission.

4 FPGA Ethernet Processing and Trading FSM Implementation

The core of our system is implemented in Verilog on the DE2-115 FPGA using a modified version of an open-source Ethernet repeater. This module was extended to include packet parsing, filtering, and conditional order transmission logic.



4.1 System Architecture

The FPGA design consists of three primary components:

- **Ethernet PHY Interface**

Handles low-level communication with the Ethernet hardware using RGMII. This includes:

- DDR input/output for receive and transmit paths

- Clock domain management (125 MHz, 25 MHz, 2.5 MHz)
 - PHY configuration via MDIO
- **Packet Buffering System**

Incoming packets are:

- Stored in RAM buffers
 - Indexed via FIFO queues
 - Accessed byte-by-byte for parsing
- **Custom Trading FSM**

A finite state machine processes incoming packets and determines whether to send a trading order.

These components are designed to handle parsing in parallel to one another to allow the processing of the Ethernet and UDP headers to happen within 1 clock cycle.

4.2 Custom Packet Transmission

A pre-constructed **UDP buy order packet** is stored directly in FPGA memory.

This packet includes:

- Ethernet header (MAC addresses)
- IPv4 header (with precomputed checksum)
- UDP header
- Custom exchange order payload

Key characteristics:

- Hardcoded destination MAC and IP (exchange)
- Source MAC set to FPGA

- UDP port configured for exchange communication
- Payload encodes a **BUY order** with:
 - Symbol = 1
 - Quantity = 20
 - Price = 100

This approach avoids the need for dynamic packet construction and simplifies transmission. In the future, we will implement a dynamic order table that allows a variety of prebuilt orders to be sent at varying quantities and prices.

4.3 Packet Filtering Pipeline

Incoming packets are processed using a strict sequential filtering implemented as an FSM. Each stage ensures that only valid market data triggers a trade.

Filtering Steps

1. **FIFO Reception**
 - Wait for a packet to arrive
 - Extract buffer index and packet length
2. **Magic Number Check**
 - Verify bytes match "GOIRISH!"
 - Confirms packet originates from expected source
3. **Message Type Check**
 - Ensure message type equals TRADE_SUMMARY (5):

```
◦ struct trade_summary {  
◦     md_header header;  
◦     Symbol symbol;  
◦     SIDE aggressor_side;
```

- Quantity total_quantity;
- Price last_price;
- } __attribute__((packed));

4. Price Extraction

- Read 4 bytes from payload
- Reconstruct 32-bit little-endian integer

5. Trading Condition

- Check if price > 10
- Ensure price is positive

Only if all conditions are satisfied does the system proceed to transmit a buy order:

```
struct new_order {  
    md_header header;  
    OrderId order_id;  
    Symbol symbol;  
    SIDE side;  
    Quantity quantity;  
    Price price;  
    uint8_t flags;  
} __attribute__((packed));
```

4.4 FSM Design

The trading logic is implemented as a finite state machine with the following states:

- **AWAIT_FIFO**
Wait for incoming packet
- **GET_FIFO**
Retrieve packet metadata
- **CHECK_MAGIC / WAIT**
Validate packet signature
- **CHECK_MSGTYPE / WAIT**
Confirm packet type
- **CHECK_PRICE / WAIT**
Extract and evaluate price
- **QUEUE_SEND**
Push prebuilt order into transmit FIFO
- **SEND_WAIT**
Ensure proper transmission timing
- **DISCARD**
Ignore invalid packets

The trading logic is implemented as a finite state machine consisting of several distinct states that guide the system through the process of receiving, validating, and responding to market data. The FSM begins in the **AWAIT_FIFO** state, where it waits for an incoming packet, before moving to **GET_FIFO** to retrieve the associated packet metadata. It then progresses through a series of validation states, **CHECK_MAGIC** and **CHECK_MSGTYPE**, to confirm the packet's signature and type respectively, with intermediate **WAIT** states ensuring proper timing between checks. Once the packet is confirmed to be valid, the **CHECK_PRICE** state extracts and evaluates the price field. If a trading condition is met, the FSM transitions to **QUEUE_SEND** to push a prebuilt order into the transmit FIFO, followed by

SEND_WAIT to ensure proper transmission timing. Any packets that fail validation are routed to the DISCARD state and ignored.

4.5 Memory and Data Handling

Packets are read from RAM using fixed byte offsets that correspond to specific fields within the Ethernet frame. Key offsets include byte 42, which marks the start of the UDP payload, byte 64, which holds the message type, and byte 74, which contains the price field. The price value is reconstructed byte-by-byte using little-endian accumulation, a method chosen to ensure compatibility with software-generated packets where the least significant byte is stored first. This approach allows the FPGA to reliably interpret multi-byte numerical fields without relying on higher-level protocol abstractions.

4.6 Transmission Mechanism

When a valid trading condition is detected, the FPGA initiates the transmission process by writing a prebuilt packet into a dedicated transmit buffer. The buffer index and packet length are then pushed into a TX FIFO, from which the Ethernet controller reads and handles the serialization and physical transmission of the data. This architecture deliberately decouples the decision-making logic from the transmission timing, ensuring that the trading FSM can continue operating independently without being stalled by the mechanics of sending data over the network.

4.7 Key Design Tradeoffs

Several design decisions were made due to project constraints. Our team found it impractical given the semester long time constraint to attempt to implement a full trading strategy over FPGA. We also discovered that sending a full packet over TCP connection would be too daunting to tackle due to the requirements of a TCP packet constructor component, so we decided to ask Prof. Belcher to open up a UDP port to his exchange for our project purposes with the following constraints in mind:

- **Hardcoded packets instead of dynamic construction**

Simplifies implementation but reduces flexibility (BUY order with custom sequence number, price, quantity, src/dst IP and MAC address):

```
○ // CUSTOM PACKET (HARD CODED BUY ORDER)
○ //
○ // IP header checksum for src=192.168.0.1 dst=192.168.13.100:
○ // Words: 4500 0046 0001 4000 4011 0000 C0A8 0001 C0A8 0D64
○ // Carry-wrapped sum = 540F -> one's complement = ABF0
○ //
○ localparam int CUSTOM_PKT_LEN = 84;
○ localparam logic [7:0] CUSTOM_PKT [0:CUSTOM_PKT_LEN-1] = '{
○
○ // ——— Ethernet Header (14 bytes) ———
○ // Destination MAC: exchange server
○ 8'h00, 8'h0F, 8'h53, 8'h4B, 8'hE6, 8'hB1,
○ // Source MAC: FPGA
○ 8'hDE, 8'hAD, 8'hBE, 8'hEF, 8'hCA, 8'hFE,
○ // EtherType: IPv4
○ 8'h08, 8'h00,
○
○ // ——— IPv4 Header (20 bytes) ———
○ 8'h45, // Version=4, IHL=5
○ 8'h00, // DSCP/ECN
○ 8'h00, 8'h46, // Total length = 70
○ 8'h00, 8'h01, // Identification
○ 8'h40, 8'h00, // Flags=DF, Fragment offset=0
○ 8'h40, // TTL=64
○ 8'h11, // Protocol=UDP
○ 8'hAB, 8'hF0, // Header checksum (src=192.168.0.1, dst=192.168.13.100)
○ 8'hC0, 8'hA8, 8'h00, 8'h01, // Source IP: 192.168.0.1
○ 8'hC0, 8'hA8, 8'h0D, 8'h64, // Dest IP: 192.168.13.100
○
○ // ——— UDP Header (8 bytes) ———
○ 8'hC0, 8'h00, // Source port: 49152
○ 8'h82, 8'h35, // Dest port: 33333
○ 8'h00, 8'h32, // UDP length = 50
```

```

○ 8'h00, 8'h00, // Checksum disabled
○
○ // — oe_request_header (20 bytes) —————
○ 8'h2A, 8'h00, // length = 42
○ 8'h01,      // msg_type = NEW_ORDER
○ 8'h01,      // version = 1
○ 8'h01, 8'h00, 8'h00, 8'h00, // seq_num = 1
○ 8'h07, 8'h00, 8'h00, 8'h00, // client_id = 7
○ 8'h54, 8'h32, 8'h4A, 8'h71, // session_id = "T2Jq"
○ 8'h71, 8'h6E, 8'h51, 8'h79, //      "qnQy"
○
○ // — new_order payload (22 bytes) —————
○ 8'h77, 8'h31, 8'h40, 8'h00, 8'h00, 8'h00, 8'h00, 8'h00, // order_id
○ 8'h01, 8'h00, 8'h00, 8'h00, // symbol = 1
○ 8'h01,      // side = BUY
○ 8'h14, 8'h00, 8'h00, 8'h00, // quantity = 20
○ 8'h64, 8'h00, 8'h00, 8'h00, // price = 100
○ 8'h00      // flags = NONE
○ };
○
○ localparam logic [2:0] CUSTOM_PKT_BUF = 3'd7;
○ logic [6:0] cust_init_idx = '0; // 7 bits to cover indices 0-83

```

○

- **Open-source Ethernet stack instead of Intel IP**

[LispEngineer's Ethernet Repeater](#) is an open source Ethernet library that reflects back incoming ethernet packets over RX. This allowed our group to avoid licensing issues that came with using Intel's configurable Ethernet parsing libraries but increased development complexity of our project.

4.8 Result

This implementation successfully demonstrates real-time packet parsing in hardware, deterministic trading decision logic, and end-to-end Ethernet-based order transmission. By integrating packet reception,

This validated the full pipeline from packet ingestion to decision-making and transmission. However, due to time constraints, we were not able to fully verify whether the FPGA's transmitted orders were successfully registered by the exchange through the physical server infrastructure. In principle, as long as the correct UDP destination port and destination MAC address are known, the FPGA should be capable of directly interfacing with the exchange over Ethernet, enabling full participation in the trading system.

5 Demonstration Setup

To test and demonstrate the system, we developed a Python-based tool that simulates market data being transmitted over Ethernet. This approach allowed us to isolate trade summary messages from other background network traffic, such as system multicasts and unrelated packets, ensuring that our FPGA logic was being evaluated under controlled conditions. The script, `send_market_update.py`, constructs and sends UDP packets containing trade summary data, including fields such as trade price (e.g., \$15), quantity, and associated metadata. On the receiving end, the FPGA captures these packets, parses the payload in hardware, and evaluates the embedded trading condition. If the incoming trade price exceeds the defined threshold, the FPGA responds by generating and transmitting a preconfigured buy order packet over Ethernet.

To reproduce this setup, several configuration and deployment steps are required. First, the Intel Quartus development environment must be installed, and the project must be properly loaded. The FPGA is then programmed by compiling and synthesizing the design, followed by flashing it onto the DE2-115 board using Active Serial Programming. A step-by-step walkthrough of this process can be followed in this setup [tutorial](#). After programming the FPGA, proper network configuration is essential. This includes manually creating an ARP table entry on the host machine to associate the FPGA's IP address

(192.168.0.2) with its corresponding MAC address, ensuring that packets are routed correctly. Once configured, the Python script can be executed to send simulated market data to the FPGA's Ethernet interface. To verify correct system behavior, Wireshark can be used to monitor network traffic. A successful result is indicated by the transmission of a packet with a length of 84 bytes and a destination IP address of 192.168.13.100, confirming that the FPGA correctly processed the input and generated the expected buy order.

6.1 Licensing Constraints

One of the most significant challenges we faced was the requirement of an expensive private license to access Intel's advanced Ethernet IP cores. These proprietary libraries provide robust and fully integrated support for Ethernet, UDP, and TCP communication (packet building, not TCP port establishment), which would have significantly simplified development. However, they were not accessible within our environment, forcing us to explore alternative solutions. As a workaround, we utilized an open-source Ethernet repeater library, which allowed us to mirror incoming Ethernet packets and maintain visibility into packet flow. This approach also enabled us to build custom parsing logic directly on top of the incoming data stream. While functional, this solution required substantially more manual implementation and a deeper understanding of low-level Ethernet behavior compared to using prebuilt IP cores.

6.2 Hardware Issues

In addition to software and tooling challenges, we encountered several physical hardware issues that impacted development. Both the FPGA board and the host machine had unreliable or partially broken Ethernet ports, which led to intermittent connectivity problems. Michael discovered this issue after testing one of the Altera boards he received with his ethernet chord. When successfully connected to an ethernet interface, the FPGA will show the status of the ethernet connection through the following lights flashing:

RX (receive), TX (transmit), DUP (can receive and transmit in parallel), and maximum data transfer speed (1000, 100, 10 Megabits/S). Our first board's ethernet monitoring lights would not turn on when connected to Michael's laptop. These issues made it difficult to determine whether failures were due to bugs in our design or simply hardware malfunctions. As a result, a significant amount of time was spent troubleshooting connectivity by testing across multiple machines, swapping Ethernet cables and interfaces, and verifying packet transmission using packet sniffing tools. This process, while time-consuming, was essential for isolating and resolving issues that were not immediately apparent from the FPGA design alone.

6.3 Network Configuration (ARP)

Another major challenge involved properly configuring network communication between the FPGA and the intended exchange endpoint. In particular, the FPGA required a valid ARP table entry in order to correctly map IP addresses to MAC addresses. Both the FPGA's IP address and the destination IP address (representing the exchange) needed to be manually configured to ensure proper routing of packets. Without this configuration, transmitted packets would fail to reach their destination, even if they were correctly constructed. Our group resolved this by manually entering in ARP entries which mapped our ethernet interface's IP (final project uses NDFEX ethernet IP), to the arbitrary IP of the FPGA to avoid having to send out ARP requests to a device that can't send back ARP messages (our FPGA). This experience underscored the importance of understanding low-level networking concepts, such as ARP and Ethernet addressing, when developing FPGA-based systems that interact directly with network infrastructure.

7 UDP Packet Handling

7.1 Market Data Deconstruction

Incoming UDP packets were parsed to extract:

- Trade price
- Message type
- Relevant payload fields

7.2 UDP Payload Construction

For outgoing orders, the FPGA constructs custom UDP packets:

- Headers are manually assembled
- Payload is formatted to match expected exchange structure
- Data fields (price, order type) are inserted based on strategy
 - BUY symbol 1 @ 20 for \$100

This process ensures compatibility with exchange-style communication.

8 Future Work

Exchange Integration

We should directly integrate our FPGA with a live exchange environment via Professor Belcher's switch for his trading server. In the future, we will bypass the need to create an ARP cache by creating an internal ARP table that automatically stores 48 bit MAC addresses. This requires establishing low-level network connectivity by identifying the correct UDP port mappings and resolving the destination MAC address associated with the exchange interface for proper Layer 2 frame delivery. We will also have to implement some sort of TCP header handling along with figuring out TCP port establishment over software. End-to-end validation is also required which involves confirming that FPGA-generated order messages are correctly parsed, accepted, and acknowledged by the exchange system, which may require packet capture (e.g., via Wireshark) and protocol-level debugging to verify correctness and timing.

Strategy Improvements

In the future, we can implement specific aspects of market trading that can be executed by several FPGAs. Right now, we only send out a hard-coded BUY order whenever we receive a trade summary above a fixed price. In the future, the FPGA will be more robust in terms of what it sends by having an internal table of hardcoded orders. Additionally, exposing configurable parameters (e.g., thresholds, order sizes, timing intervals) via registers or a control interface enables runtime adaptability without requiring synthesis, improving flexibility for testing and deployment.

System Enhancements

At the system level, replacing the existing open-source Ethernet repeater with a fully custom hardware Ethernet stack would allow for control over packet parsing, header generation, and transmission timing, reducing latency and eliminating unnecessary buffering. Reliability can be improved through better clock domain crossing design, error handling, and signal integrity considerations in the hardware setup. Automating ARP resolution and network configuration within the FPGA, potentially via a lightweight ARP cache and state machine, would remove manual intervention and support dynamic network environments. Finally, optimizing the end-to-end pipeline for latency and throughput, including minimizing combinational path delays and maximizing pipeline parallelism, is essential for achieving competitive performance in high-frequency trading applications. Additionally, we can reduce the number of clock cycles needed to parse the UDP payload by changing the sequential processing of the packet checks with a parallel check system involving independent order checking modules (CRC, MSG_TYPE, quantity, price). We should also be able to smartly throw away orders through immediately tossing any market data within the first few clock cycles of reading by checking the first 4 bytes of each UDP payload.

9 Conclusion

This project successfully demonstrates the feasibility of implementing a hardware-based trading system on the DE2-115 FPGA. By parsing real-time market data and generating trading decisions entirely in hardware, we highlight the potential speed advantages of FPGA-based trading systems.

Despite challenges related to licensing, hardware limitations, and networking complexity, we developed a functional prototype that lays the groundwork for future exploration. With further development, this system could evolve into a fully integrated FPGA trading platform capable of interfacing directly with live exchanges.